第一章 資料結構簡介

大綱

- 1.1 前言
- 1.2 演算法 (Algorithm)
- 1.3 演算法的效率評估
- 1.4 資料表示法
  - 1.4.1 常見的幾種進制表示法
  - 1.4.2 整數資料表示
  - 1.4.3 實數表示法 1.4.4 文數資料表示法
  - 1.4.4.1 ASCII 碼
  - 1.4.4.2 EBCDIC 碼
- 1.5 C 語言重點回顧
  - 1.5.1 迴圈控制敘述
  - 1.5.2 條件判斷結構
- 1.6 常見的資料結構
- 習題

1.1 前言

- 資料 (Data): 要讓電腦處理的原始內容
- 資訊 (Information): 電腦處理過後所產生的有意義訊息



圖 1.1 資料與資訊的關係

1.1 前言 – 續

- 利用電腦來處理資料有兩種方式：

1. 利用事先設計好的套裝軟體來進行資料處理的動作，使用者所扮演的角色是應用程式使用者 (Application User). 例如微軟的 Word ，我們就可以透過它來處理文書資料 .

2. 要自行撰寫程式來解決給定的問題，使用者所扮演的角色是程式撰寫者 (Programmer).

1.1 前言 – 續

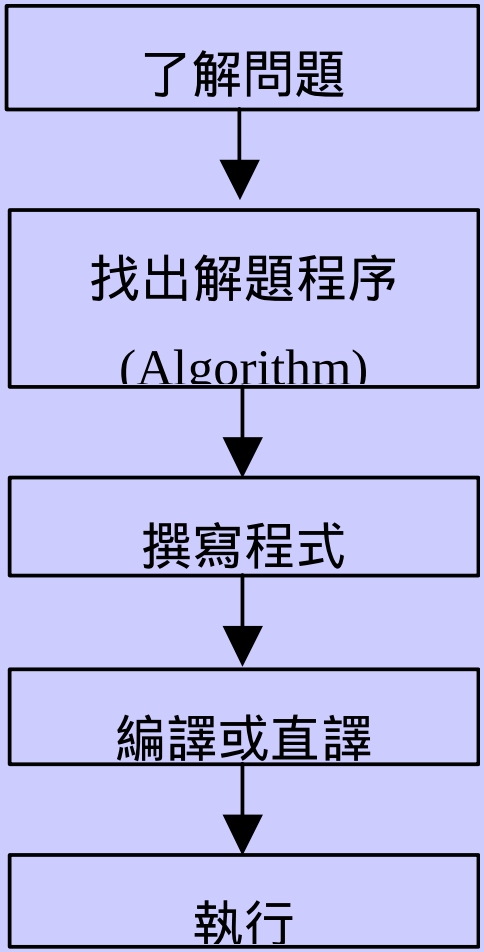


圖 1.2 解決問題的流程

1.1 前言 – 續

1. 先了解問題的定義並找出要處理的資料
2. 將解題的程序也就是演算法 (Algorithm) 找出來並詳細的加以列出
3. 挑選一個合適的程式語言 (Programming Language) 來撰寫程式，在本書中我們是以 C 語言來撰寫程式
4. 將寫好的程式透過編譯或直譯的方式來轉換成可執行的機器碼 (Executable Machine Code)
5. 如果解題的程序是正確的，則我們所希望獲得的資訊會在機器碼執行後正確地產生

1.1 前言 – 續

- 資料結構 (Data Structure) 與整個利用電腦來處理問題的關係究竟為何？
  - 資料結構是我們將解題程序撰寫成對應程式的過程中用來表示處理資料的方法
  - 表示相同資料的方法並不是唯一的，但是若能妥善利用好的資料結構來表示處理的資料，通常能夠使得程式執行的效率提升或是減少程式所需的變數儲存空間

1.2 演算法

- 演算法是為了解決一個給定的問題，經過問題的分析與了解後所撰寫出來的一系列解題程序
- 一個演算法必須滿足下列五個條件：

1. 輸入資料：一個演算法所須之輸入資料可有可無
2. 輸出資料：一個演算法至少必須有一個輸出結果
3. 有限性：亦即須能在有限個步驟之內解決問題
4. 明確性：即每一個步驟必須條理分明，意義清楚
5. 有效性：整個解題程序執行完後必須能夠解決給定的問題。也就是能夠輸出對應的資訊

1.2 演算法 – 續

- 描述一個演算法常見的方式有兩種：
  1. 虛擬語言 (Pseudo Language): 是一種用來描述解題程序的語法，並不是屬於真正的程式語言而是一種接近英文語法的描述方式
  2. 流程圖 (Flow Char) 的繪製 : 利用事先定義好的一些圖形與流程控制圖示來描述整個問題的解題程序

1.2 演算法 – 續

表 1.1 虛擬語言的介紹

基本元素	意義	範例
INPUT	從使用者處輸入資料	INPUT radius
OUTPUT	輸出資料給使用者	OUTPUT area
=	指定等號右邊計算完成的結果給等號左方的變數	area=PI*radius*radius
STOP	結束演算法	STOP
IF ELSE	當條件滿足便執行第一段敘述，否則執行第二段敘述	IF grade < 60 OUTPUT fail ELSE OUTPUT success
LOOP	重複執行多次給定敘述	LOOP num times INPUT grade1,grade2 OUTPUT (grade1+grade2)/2
WHILE	當條件滿足時執行給定敘述	WHILE classno<0 OUTPUT error INPUT classno

1.2 演算法 – 續

計算梯形的面積，其中梯形的上底、下底與高度是由使用者所輸入。而梯形面積的計算公式為 (上底 + 下底)* 高度 /2 。

【演算法】計算梯形面積

```
STEP 1: INPUT base1, base2, height
STEP 2: basenum=base1+base2
STEP 3: area=basenum*height/2
STEP 4: OUTPUT area
STEP 5: STOP
```

1.2 演算法 – 續

【程式範例】計算梯形面積

```
 1 /*********************************************************/
 2 /*【程式名稱】: 1_area_of_trapezoid.c                     */
 3 /*【程式功能】: 計算梯形面積                              */
 4 /*********************************************************/
 5 /*【變數名稱及用途】                                      */
 6 /* area: 用來記錄算出的梯形面積                          */
 7 /*********************************************************/
 8
 9
10 #include <stdio.h>
11 main()
12 {
13   int base1, base2, height;
14   int basenum, area;
15   scanf("%d",&base1);
16   scanf("%d",&base2);
17   scanf("%d",&height);
18   basenum = base1 + base2;
19   area = basenum * height / 2;
20   printf("\nThe area is %d",area);
21 }
```

1.2 演算法 – 續

讀入三位學生的資料結構成績，並計算其平均分數。要算出三位學生的平均分數可先算出成績的總和後除以人數。

【演算法】計算三位同學的平均分數

```
STEP 1: INPUT grade1, grade2, grade3
STEP 2: sum=grade1+grade2+grade3
STEP 3: average=sum/3
STEP 4: OUTPUT average
STEP 5: STOP
```

1.2 演算法 – 續

【程式範例】計算三位同學的平均分數

```
 1 /*********************************************************/
 2 /*【程式名稱】: 1_aveage_grade1.c                         */
 3 /*【程式功能】: 計算三位同學的平均分數                    */
 4 /*********************************************************/
 5 /*【變數名稱及用途】                                     */
 6 /* average: 用來記錄算出的平均分數                       */
 7 /*********************************************************/
 8
 9
10 #include <stdio.h>
11
12  main()
13 {
14  int grade1, grade2, grade3;
15  int sum, average;
16  scanf("%d",&grade1);
17  scanf("%d",&grade2);
18  scanf("%d",&grade3);
19  sum=grade1+grade2+grade3;
20  average=sum/3;
21  printf("\nThe average score = %d", average);
22 }
```

1.2 演算法 – 續

讀入三個數值 A ， B ， C ，並找出其中最大數值。

【演算法】找出三個數值中的最大值

```
STEP 1: INPUT A, B, C
STEP 2: IF (A>B)
          IF (A>C)                    /* A > B and A > C → MAX=A */
              MAX= A
          ELSE                        /* A > B and C > A → MAX=C */
              MAX = C
        ELSE
          IF (B>C)                    /* B > A and B > C → MAX=B */
              MAX = B;
          ELSE                       /* B > A and C > B → MAX = C */
              MAX = C;
STEP 3: OUTPUT MAX
STEP 4: STOP
```

1.2 演算法 – 續

【程式範例】找出三個數值中的最大值

```
 1 /*********************************************************/
 2 /*【程式名稱】: 1_find_maximum.c                          */
 3 /*【程式功能】: 找出三個數值中的最大值                    */
 4 /*********************************************************/
 5 /*【變數名稱及用途】                                      */
 6 /* MAX: 用來記錄算出的最大值                             */
 7 /*********************************************************/
 8
 9
10 #include <stdio.h>
11
12 main()
13 {
14  int A, B, C;
15  int MAX;
16  scanf("%d",&A);
17  scanf("%d",&B);
18  scanf("%d",&C);
19  if (A>B)
20     {
21       if (A>C)
22          MAX=A;
23       else
24          MAX=C;
25     }
26  else
27    {
28       if (B>C)
29          MAX=B;
30       else
31          MAX=C;
32    }
33   printf("\nThe maximal number is = %d", MAX);
34 }
```

1.2 演算法 – 續

利用數學的描述法，求 n 階層 (n!) 之值，可以寫成： n! = n * (n-1)*… * 1 ， 0! = 1

【演算法】計算 n!

```
STEP 1: INPUT n
STEP 2: product = 1
STEP 3: multiplier=n
STEP 4: WHILE (multiplier>1)
      a. product=product*multiplier
      b. multiplier=multiplier-1
STEP 5: OUTPUT product
STEP 6: STOP
```

1.2 演算法 – 續

【程式範例】計算 n!

```
 1 /*********************************************************/
 2 /*【程式名稱】: 1_n_factorial.c                           */
 3 /*【程式功能】: 計算 n!                                   */
 4 /*********************************************************/
 5 /*【變數名稱及用途】                                      */
 6 /* product: 用來記錄算出的 n!的結果                       */
 7 /*********************************************************/
 8
 9
10 #include <stdio.h>
11
12 main()
13 {
14   int n;
15   int product=1, multiplier;
16   scanf("%d",&n);
17   multiplier = n;
18   while ( multiplier >1 )
19     {
20       product *= multiplier;
21       multiplier--;
22     }
23   printf("\nThe result of %d factorial = %d",n, product);
24 }
```

1.3 演算法的效率評估

- 演算法的效率指的是計算根據該演算法所撰寫的程式在經過編譯後實際執行所需的時間

- 有可能因為程式編譯的過程或是電腦設備的差異使得效率分析會因電腦的軟硬體不同而有不同的結果

1.3 演算法的效率評估 – 續

- 除了實際衡量程式的執行時間外，另外一種效益評估的方式則是透過分析個別程式的複雜度 (Complexity)

- 通常程式的複雜度分析可以分成兩大類：
  1. 時間複雜度 (Time Complexity)
  2. 空間複雜度 (Space Complexity)

1.3 演算法的效率評估 – 續

- 時間複雜度是希望能夠透過找出程式編譯的時間與執行所需的時間來評估其效率

- 幾種可能影響程式執行的因素列於下：
  1. 電腦的處理速度
  2. 機器的指令集 (Instruction Set)
  3. 編譯器 (Compiler)

1.3 演算法的效率評估 – 續

- 為了簡化時間複雜度的分析，我們只著重於程式必須執行的指令個數而不去考慮個別指令實際執行所需的時間

程式範例	可否執行	頻率	步驟計數
`int sum(int grades[],int n)`	N	0	0
`{`	N	0	0
`  int i;`	N	0	0
`  int tempsum=0;`	Y	1	1
`  for(i=0;i<n;i++)`	Y	n+1	n+1
`    {`	N	0	0
`      tempsum+=grades[i];`	Y	n	n
`    }`	N	0	0
`  return tempsum;`	Y	1	1
`}`	N	0	0
Total			2n+3

1 .3 演算法的效率評估 – 續

- 理論上我們通常將一個程式 P 的時間複雜度表示成 T(P) 的形式，而 T(P) 記錄了程式的實體特性 n 的成長速率

- 上面所介紹的程式片段所需的時間複雜度 T(P)=2n+3

- 請記得實際上的 T(P) 應該還要加上程式的編譯時間

1 .3 演算法的效率評估 – 續

- 常見的時間複雜度函數可能為下列形式：
  1. T(P)=c, 為一常數多項式
  2. T(P)=an+b, 其中 a>0,b³0 為一個線性多項式
  3. T(P)=an2+bn+c, 其中 a>0 為一個二次方多項式
  4. T(P)=an3+bn2+cn+d, 其中 a>0 為一個三次方多項式
  5. T(P)=an, 其中 a>0 為一個指數多項式
  6. T(P)=alogn+b, 其中 a>0 為一個對數多項式

1 .3 演算法的效率評估 – 續

- 當輸入資料的個數 n 值我們可以得到下列關係 ：

  logn < n < nlogn < n2 < n3 < 2n

表 1.2 常見的多項式成長速率比較

	logn	n	nlogn	n^2	n^3	2^n
10	1.00	10	10.00	100	1000	1024
20	1.30	20	26.02	400	8000	1048576
30	1.48	30	44.31	900	27000	1073741824
40	1.60	40	64.08	1600	64000	1.09951E+12
50	1.70	50	84.95	2500	125000	1.1259E+15
60	1.78	60	106.69	3600	216000	1.15292E+18
70	1.85	70	129.16	4900	343000	1.18059E+21
80	1.90	80	152.25	6400	512000	1.20893E+24
90	1.95	90	175.88	8100	729000	1.23794E+27
100	2.00	100	200.00	10000	1000000	1.26765E+30

1.3 演算法的效率評估 – 續

- 一個程式 P 的空間複雜度 (Space Complexity) 包含固定的空間需求與變動的空間需求兩個部分
- 固定的空間需求包含了程式在編譯時期 (Compile Time) 所產生的空間需求
- 變動空間需求包含了動態記憶體要求 (Dynamic Memory Allocation) 所配置的空間，以及遞迴函數執行時所需要用來儲存活動紀錄 (Activation Record) 的執行時堆疊空間 (Run Time Stack)

1 .3 演算法的效率評估 – 續

- 通常我們會利用 S(P) 來表示一個程式 P 所需的空間需求，而 S(P)=c+SP(I)
- c 代表這個程式的固定空間需求
- SP(I) 則表示程式 P 針對特定的輸入資料集合 I (Instance) 所需的變動空間需求

1.4 資料表示法

- 數位化資料可分成兩類：
  1. 數值資料：可進行加、減、乘、除等算術運算的資料
  2. 文數資料：不能拿來運算的資料

- 常見的數值表示法可以分成兩大類：整數與實數

- 整數與實數最大的差別是在於實數能夠表示包含小數的數值資料

1 .4.1 常見的幾種進制表示法

表 1.3 不同進制間的轉換範例

十進制 (Decimal)	二進制 (Bianry)	八進制 (Octal)	十六進制 (Hexadecimal)
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10

1.4.2 整數資料表示 – 續

- 常見的整數表示法有兩類：
  1. 無正負號之整數 (Unsigned Integer) :
     只能表示非負的數值，而其能表示的數字範圍跟儲存空間的大小有關
  2. 整數 (Integer) : 包含了正整數、零與負整數

- 利用二進制來表示整數資料的做法中，常見的表示方式有三種 ：
  1. 符號帶大小 (Sign Significant) 表示法
  2. 1 的補數 (1’s Complement) 表示法
  3. 2 的補數 (2’s Complement) 表示法

1 .4.2 整數資料表示 – 續

- 符號帶大小的表示法 :

  是利用一個位元來記錄數值的正負號資訊 (Sign) ，通常表示負整數時其對應的符號位元為 1 ，反之則為 0

- 1 的補數表示法 :

  將數值之二進制中的 0 變為 1 且將 1 變為 0

- 2 的補數表示法 :

  將此數值對應的 1 的補數計算出後再將結果加 1 就可得到其對應 2 的補數的結果

- 電腦內部用來表示一般整數的方法是利用 2 的補數

1 .4.2 整數資料表示 – 續

表1.4 利用四個位元表示一般的整數

十進制	符號帶大小	1 的補數	2 的補數
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	1000	1111	0000
-1	1001	1110	1111
-2	1010	1101	1110
-3	1011	1100	1101
-4	1100	1011	1100
-5	1101	1010	1011
-6	1110	1001	1010
-7	1111	1000	1001
-8	N/A	N/A	1000

1.4.2 整數資料表示 – 續

表1.5 整數的種類與能夠表示的數值範圍

	種類	儲存空間	數值範圍
無正負號整數	短無正負號整數	16 bits	0 至 2^{15}-1
	長無正負號整數	32 bits	0 至 2^{32}-1
一般整數	短整數	16 bits	-2^{15} 至 2^{15}-1
	長整數	32 bits	-2^{31} 至 2^{31}-1

1.4.2 整數資料表示 – 續

【程式範例】列印不同型態整數的儲存空間

```
 1 /*************************************************************/
 2 /*【程式名稱】: 1_storage_size.c                              */
 3 /*【程式功能】: 找出不同型態的整數儲存空間                    */
 4 /*************************************************************/
 5
 6 #include <stdio.h>
 7
 8 main()
 9 {
10  unsigned short int a;
11  unsigned int       b;
12  short int          c;
13  int                d;
14
15  printf("\nSize of unsigned short int = %d bytes",sizeof(a));
16  printf("\nSize of unsigned int = %d bytes",sizeof(b));
17
18  printf("\nSize of short int = %d bytes",sizeof(c));
19  printf("\nSize of int = %d bytes",sizeof(d));
20  printf("\n");
21
22 }
```

1 .4.3 實數表示法

- 實數的組成包含三個部分：符號位元 (Sign Bit) 、指數部分 (Exponent) 與假數部分 (Mantissa)

- 符號位元則是用來表示正負號，而指數部分與假數部分是用來紀錄實數在轉換成二進制科學表示法後的資訊

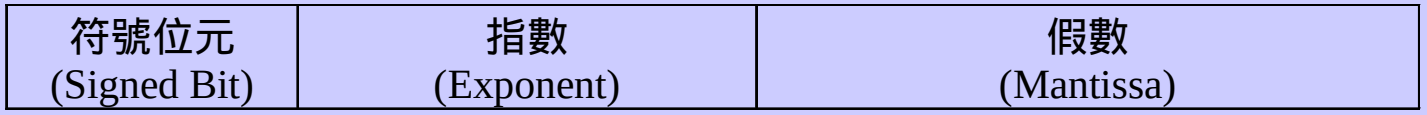


圖 1.3　浮點數值的三個組成部分

1 .4.3 實數表示法 – 續

- 要表示一個給定的實數時，我們會先將其轉換成對應的二進制表示法，接著再轉換成科學表示法。也就是轉換成 $(-1)^s \times m^s \times 2^e$ 的形式

- m 的數值代表給定的實數的假數部分的數值，將 m 的小數點後所有其他的數值紀錄到假數的部分即可完成假數部分的處理

1.4.3 實數表示法 – 續

- e 的數值代表給定的實數的指數次方的數值，大多數的電腦對於指數數值是以 $e+2^{t-1}$ 的方式來表示之，其中 t 代表指數部份的總位元數
- 為了要利用 t 個位元來表示所有可能的狀況通常採用的方式是透過平移原始的指數數值來處理

表 1.6 利用八個位元紀錄指數數值之平移處理

指數數值 e	-128	-127	…	-1	0	1	…	126	127
對應二進制數值	0	1	…	127	128	129	…	255	256

1 .4.3 實數表示法 – 續

- 實數資料在電腦中有兩種常見的表示法，即短浮點式 (Short Floating Point Format) 及長浮點式 (Long Floating Point Format)

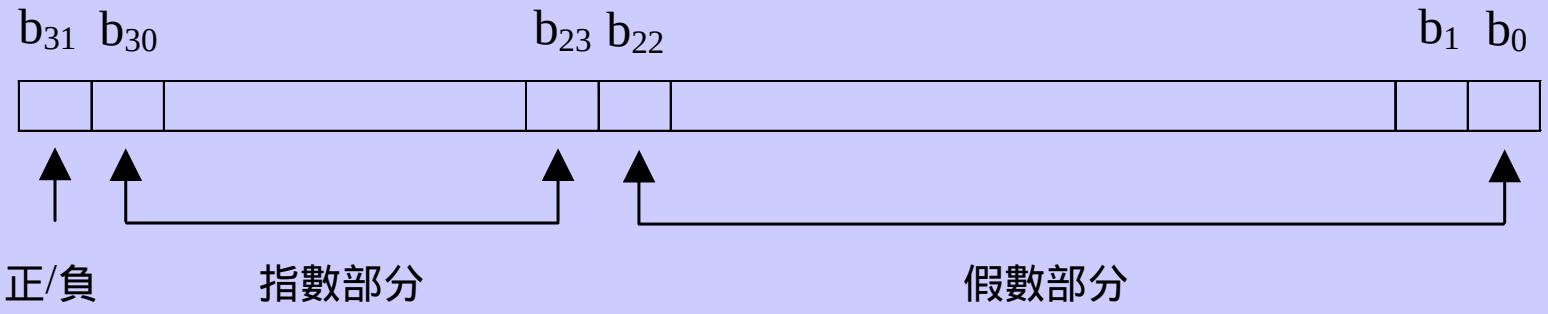


圖 1.4 IEEE Standard 754 之 32 位元短浮點式表示法

1 .4.3 實數表示法 – 續

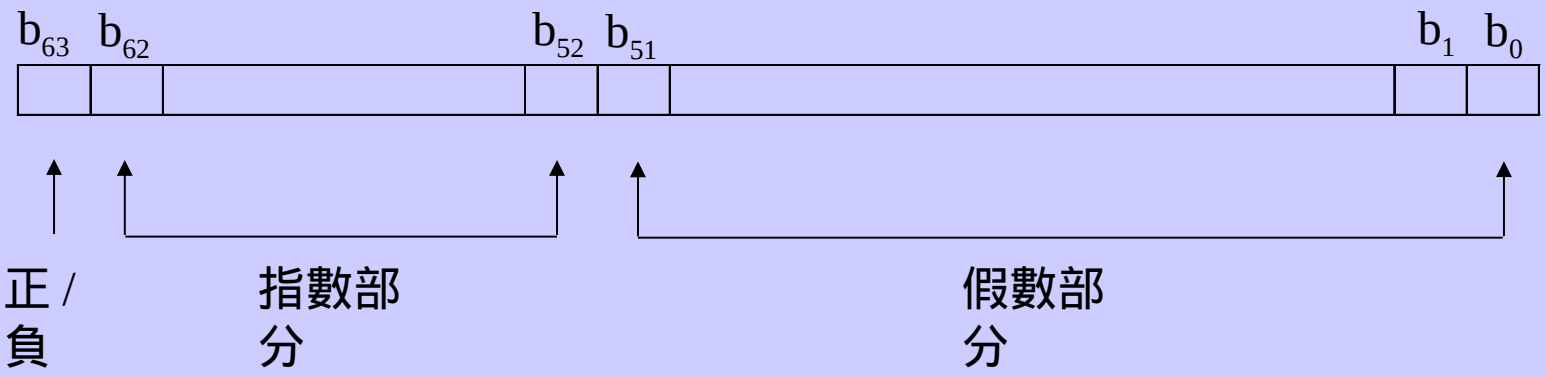


圖 1.4 IEEE Standard 754 之 64 位元常浮點式表示法

1.4.4 文數資料表示法

- 文數資料 (Alphanumeric Data) 是包含文字 (Letters)、符號 (Symbols) 與數字 (Digits) 的資料，所有不可做算術運算的資料皆屬此類
- 目前電腦中最常使用的文數資料表示法有兩種 :

1. ASCII 碼 (American Standard Code for Information Interchange, 美國標準資訊交換碼)，目前市面上個人電腦通用的資料表示法
2. EBCDIC 碼 (Extended Binary Coded Decimal Interchange Code) ，EBCDIC 碼是 IBM 、 UNIVAC 等一些大型電腦所採用的編碼法

1 .4 .4 .1 ASCII 碼

- ASCII 碼是由美國國家標準局 (ANSI) 所制定的
- 早期的 ASCII 碼由七個位元來表示一個字元 (Character) ，總共可以表示 128 種不同的符號，其中包含大小寫英文字母、阿拉伯數字、標點符號與四則運算符號等鍵盤上常看到的按鍵
- 剩餘的組合可用來表示一些特殊符號及螢幕與列表機的控制符號
- 隨著電腦軟硬體的快速發展，七位元的 ASCII 碼被擴充成八位元的 ASCII 碼

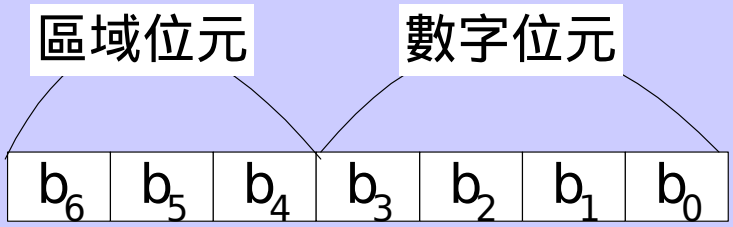


圖 1.7 七位元 ASCII 之排列方式

1 .4 .4 .1 ASCII 碼 – 續

表 1.7 七位元 ASCII 碼一覽表

字元	ASCII 碼		字元	ASCII 碼		字元	ASCII 碼		字元	ASCII 碼	
	二進位	十六進位		二進位	十六進位		二進位	十六進位		二進位	十六進位
空白	0100000	20	6	0110110	36	L	1001100	4C	e	1100101	65
!	0100001	21	7	0110111	37	M	1001101	4D	f	1100110	66
”	0100010	22	8	0111000	38	N	1001110	4E	g	1100111	67
#	0100011	23	9	0111001	39	O	1001111	4F	h	1101000	68
$	0100100	24	:	0111010	3A	P	1010000	50	i	1101001	69
%	0100101	25	;	0111011	3B	Q	1010001	51	j	1101010	6A
&	0100110	26	<	0111100	3C	R	1010010	52	k	1101011	6B
?	0100111	27	=	0111101	3D	S	1010011	53	l	1101100	6C
(	0101000	28	>	0111110	3E	T	1010100	54	m	1101101	6D
)	0101001	29	?	0111111	3F	U	1010101	55	n	1101110	6E
*	0101010	2A	@	1000000	40	V	1010110	56	o	1101111	6F
+	0101011	2B	A	1000001	41	W	1010111	57	p	1110000	70
,	0101100	2C	B	1000010	42	X	1011000	58	q	1110001	71
-	0101101	2D	C	1000011	43	Y	1011001	59	r	1110010	72
.	0101110	2E	D	1000100	44	Z	1011010	5A	s	1110011	73
/	0101111	2F	E	1000101	45	[	1011011	5B	t	1110100	74
0	0110000	30	F	1000110	46	`	1011100	5C	u	1110101	75
1	0110001	31	G	1000111	47	]	1011101	5D	v	1110110	76
2	0110010	32	H	1001000	48	a	1100001	61	w	1110111	77
3	0110011	33	I	1001001	49	b	1100010	62	x	1111000	78
4	0110100	34	J	1001010	4A	c	1100011	63	y	1111001	79
5	0110101	35	K	1001011	4B	d	1100100	64	z	1111010	7A

1 .4 .4 .1 ASCII 碼 – 續

表 1.8 部分 8- 位元的 ASCII 碼

字元	八位元 ASCII 碼	字元	八位元 ASCII 碼	字元	八位元 ASCII 碼
0	0101 0000	C	1100 0011	O	1100 1111
1	0101 0001	D	1100 0100	P	1101 0000
2	0101 0010	E	1100 0101	Q	1101 0001
3	0101 0011	F	1100 0110	R	1101 0010
4	0101 0100	G	1100 0111	S	1101 0011
5	0101 0101	H	1100 1000	T	1101 0100
6	0101 0110	I	1100 1001	U	1101 0101
7	0101 0111	J	1100 1010	V	1101 0110
8	0101 1000	K	1100 1011	W	1101 0111
9	0101 1001	L	1100 1100	X	1101 1000
A	1100 0001	M	1100 1101	Y	1101 1001
B	1100 0010	N	1100 1110	Z	1101 1010

1 .4 .4 .2 EBCDIC 碼

- 由八個位元來表示一個符號，總共可表示 256 種不同的字元

表 1.9 EBCDIC 碼一覽表

字元	二進位	十六進位	字元	二進位	十六進位	字元	二進位	十六進位	字元	二進位	十六進位
空白	01000000	40	”	01111111	7F	v	10100101	A5	R	11001001	D9
·	01001011	4B	a	10000001	81	w	10100110	A6	S	11100010	E2
<	01001100	4C	b	10000010	82	x	10100111	A7	T	11100011	E3
(	01001101	4D	c	10000011	83	y	10101000	A8	U	11100100	E4
+	01001110	4E	d	10000100	84	z	10101001	A9	V	11100101	E5
&	01010000	50	e	10000101	85	A	11000001	C1	W	11100110	E6
$	01011011	5B	f	10000110	86	B	11000010	C2	X	11100111	E7
	01011100	5C	g	10000111	87	C	11000011	C3	Y	11101000	E8
)	01011101	5D	h	10001000	88	D	11000100	C4	Z	11101001	E9
;	01011110	5E	i	10001001	89	E	11000101	C5	0	11110000	F0
¬	01011111	5F	j	10010001	91	F	11000110	C6	1	11110001	F1
-	01100000	60	k	10010010	92	G	11000111	C7	2	11110010	F2
/	01100001	61	l	10010011	93	H	11001000	C8	3	11110110	F3
,	01101011	6B	m	10010100	94	I	11001001	C9	4	11110100	F4
%	01101100	6C	n	10010101	95	J	11010001	D1	5	11110101	F5
>	01101110	6E	o	10010110	96	K	11010010	D2	6	11110110	F6
?	01101111	6F	p	10010111	97	L	11000011	D3	7	11110111	F7
:	01111010	7A	q	10011000	98	M	11000100	D4	8	11111000	F8
#	01111011	7B	r	10011001	99	N	11000101	D5	9	11111001	F9
@	01111100	7C	s	10100010	A2	O	11000110	D6			
?	01111101	7D	t	10100011	A3	P	11000111	D7			
=	01111110	7E	u	10100100	A4	Q	11001000	D8			

1 .5　C 語言重點回顧

- 1.5.1 介紹 C 語言常見的迴圈控制結構，內容包含回顧 for, while 與 do while 等迴圈控制結構

- 1.5.2 介紹 C 語言的條件判斷結構， 內容包含 if, if else 與 switch case 等條件判斷結構

1 .5 .1　迴圈控制敘述

- for 的迴圈控制結構，其對應的 C 語言語法如下所列：

```
for(起始設定敘述;條件判斷敘述;條件控制敘述)
    {
        所要執行的所有指令;
    }
```

1.5.1 迴圈控制敘述 – 續

【程式範例】利用 for loop 計算 1+2+3+…+100 的總和

```
 1 /*************************************************************/
 2 /*【程式名稱】: 1_sum1.c                                      */
 3 /*【程式功能】: 計算 1+2+3+…+100 的總和                       */
 4 /*************************************************************/
 6 /*【變數名稱及用途】                                          */
 7 /* sum: 用來記錄加總的總和                                   */
 8 /*************************************************************/
 9
10 #include <stdio.h>
11
12 main()
13 {
14   int i;
15   int sum=0;
16
17   for(i=1;i<=100;i++)
18      {
19        sum+=i;
20      }
21
22    printf("\nThe summation of 1 to 100 is = %d", sum);
23
24 }
```

1 .5 .1　迴圈控制敘述 – 續

- while 的迴圈控制結構，其對應的 C 語言語法如下所列：

```
while(條件控制敘述)
    {
        所要執行的所有指令;
    }
```

1.5.1 迴圈控制敘述 – 續

【程式範例】利用 while loop 計算 1+2+3+…+100 的總和

```
 1 /*****************************************************************/
 2 /*【程式名稱】: 1_sum2.c                                          */
 3 /*【程式功能】: 計算 1+2+3+…+100 的總和                           */
 4 /*****************************************************************/
 6 /*【變數名稱及用途】                                              */
 7 /* sum: 用來記錄加總的總和                                        */
 8 /*****************************************************************/
 9
10 #include <stdio.h>
11
12 main()
13 {
14   int i=1;
15   int sum=0;
16
17   while(i<=100)
18      {
19        sum+=i;
20        i++;
21      }
22
23    printf("\nThe summation of 1 to 100 is = %d", sum);
24 }
```

1 .5 .1　迴圈控制敘述 – 續

- do while 的迴圈控制結構與前面所介紹的 for 與 while 迴圈的最大差別在於前面兩個迴圈控制結構是 do while 迴圈控制結構至少會執行一次迴圈內所有指令
- do while 的迴圈控制結構，其對應的 C 語言語法如下所列：

```
do
   {
      所要執行的所有指令;
   }
while(條件控制敘述);
```

1.5.1 迴圈控制敘述 – 續

【程式範例】利用 do while 計算 1+2+3+…+100 的總和

```
 1 /*************************************************************/
 2 /*【程式名稱】: 1_sum3.c                                     */
 3 /*【程式功能】: 計算 1+2+3+…+100 的總和                      */
 4 /*************************************************************/
 6 /*【變數名稱及用途】                                        */
 7 /* sum: 用來記錄加總的總和                                   */
 8 /*************************************************************/
 9
10 #include <stdio.h>
11
12 main()
13 {
14   int i=1;
15   int sum=0;
16
17   do
18    {
19      sum+=i;
20       i++;
21    }
22   while (i<=100);
23
24   printf("\nThe summation of 1 to 100 is = %d", sum);
25 }
```

1 .5 .2　條件判斷結構

- if 的條件判斷結構，其對應的 C 語言語法如下所列：

```
if (條件判斷敘述)
    {
       條件成立時所要執行的所有指令;
    }
```

- 蜂巢式 if (Nested if) 的使用，也就是在一個 if 的條件判斷敘述中可以包含另一個 if 的條件判斷敘述

```
if (條件判斷敘述 1)
   {
      條件 1 成立時所要執行的所有指令;
      if (條件判斷敘述 2)
          {
             條件 2 成立時所要執行的所有指令;
          }
   }
```

1 .5 .2　條件判斷結構 – 續

- if else 的條件判斷結構，它與 if 條件判斷的差別在於當判斷條件成立或是不成立時都有對應的指令必須要執行，其對應的 C 語言語法如下所列：

```
if (條件判斷敘述)
    {
       條件成立時所要執行的所有指令;
    }
else
    {
      條件不成立時所要執行的所有指令;
    }
```

1 .5 .2　條件判斷結構 – 續

- if else (Nested if else) 條件判斷結構，其對應的 C 語言語法如下所列：

```
if (條件判斷敘述 1)
   {
      條件 1 成立時所要執行的指令;
      if (條件判斷敘述 2)
         {
            條件 2 成立時所要執行的所有指令;
         }
      else
         {
             條件 2 不成立時所要執行的所有指令;
         }
   }
else
   {
     條件 1 不成立時所要執行的指令;
      if (條件判斷敘述 3)
         {
            條件 3 成立時所要執行的所有指令;
         }
      else
         {
             條件 3 不成立時所要執行的所有指令;
         }
   }
```

1 .5 .2　條件判斷結構 – 續

- switch case 的條件判斷結構，其對應的 C 語言語法如下所列：

```
switch (條件判斷敘述)
   {
      case 條件 1:
            {   條件 1 成立所要執行的指令;          break;   }
      case 條件 2:
            {   條件 2 成立所要執行的指令;          break;   }
      case 條件 3:
            {   條件 3 成立所要執行的指令;          break;   }
        .
        .
        .
      default:
            {   所有條件皆不成立所要執行的指令;     break;   }
   }
```

1.5.2 條件判斷結構 – 續

- 在 switch 的條件判斷敘述其結果必須是整數資料或是字元資料

【程式範例】利用 switch case 算出字元出現次數

```
 1 /*************************************************************/
 2 /*【程式名稱】: 1_symbol_count.c                              */
 3 /*【程式功能】: 計算字元出現次數                              */
 4 /*************************************************************/
 5 /*【變數名稱及用途】                                         */
 6 /* no1: 用來記錄數字的出現次數                               */
 7 /* no2,no3: 用來記錄空白與其他符號的出現次數                 */
 8 /*************************************************************/
 9
10 #include <stdio.h>
11
12 main()
13 {
14   int i,ch,no1,no2,no3;
15   int sum=0;
16
17   no1=no2=no3=0;
18
```

1 .5 .2　條件判斷結構 – 續

```
19  while(  (ch=getchar()) != -1 )
20       {
21          switch(ch)
22            {
23               case ‘0’:  no1++; break;
24               case ‘1’:  no1++; break;
25               case ‘2’:  no1++; break;
26               case ‘3’:  no1++; break;
27               case ‘4’:  no1++; break;
28               case ‘5’:  no1++; break;
29               case ‘6’:  no1++; break;
30               case ‘7’:  no1++; break;
31               case ‘8’:  no1++; break;
32               case ‘9’:  no1++; break;
33               case ‘ ‘:  no2++; break;
34               case ‘\n‘: no2++; break;
35               case ‘\t‘: no2++; break;
36               default:    no3++; break;
37            }
38     }
39    printf(“\nNo of digits =%4d”,no1);
40    printf(“\nNo of white space =%4d”,no2);
41    printf(“\nNo of other symbol =%4d”,no3);
42  }
```

1.6 常見的資料結構

- 陣列 (Array)：電影院中的坐椅、排列整齊的椅子
- 鏈結串列 (Linked List)：在鐵軌上南來北往的火車，每一部火車將一節一節的車廂從頭到尾串連在一起，而連接各節車廂的連接器就是此鏈結串列的鏈結
- 堆疊 (Stack)：到西餐廳享用歐式自助餐時，餐盤一個一個依照由下而上的順序整齊的擺置
- 佇列 (Queue)：中午用餐時間，我們經常會看到學校自助餐前面大排長龍，等待享用午餐的排隊人潮
- 樹狀結構 (Tree Structure)：電腦內部檔案儲存的結構，或是歷屆學生的家族族譜，羽球比賽的單循環賽程資料等

1.6 常見的資料結構 – 續

- 圖形及網路 ： 全台三省的省道與高速公路路線圖、環島鐵路的路線圖、長榮航空公司的全球航線圖
- 排序 (Sorting)： 書店中常見的英漢字典，其中的所有英文字均是按照字母的大小次序排列的
- 赫序函數 (Hashing Function)： 利用一種技術將特定的資料例如某一個學生的學號輸入，就可以馬上找出他家裡的電話號碼、地址，…
- 檔案系統 (File System): 想要查詢某一位同學的個人資料，我們可以到檔案室中很快地找出某一位同學的學籍資料表來

1.6　常見的資料結構－續

- 學習資料結構的最主要目的：
  能妥善利用電腦將所有個人手邊的資料有系統地安排，建立資料與資料彼此間的關係，仿照日常生活中人、事、物常見的各式各樣結構，將所有的資料做最適當的安排、儲存，以方便資料的更新及存取

1.6 常見的資料結構 – 續

- 資料結構的應用主要包含下列幾個重要主題：
  1. 有效地儲存給定的各種不同資料？
  2. 不同的資料結構表示法和其相關演算法的探討
  3. 改進現存演算法的效率，使程式的執行速度趨於更快
  4. 利用合適的使用者介面來建構資料存取方法、資料儲存之結構和資料儲存之媒體
  5. 資料處理之各種技巧，比如：排序、搜尋、合併、更新、分配等演算法探討
  6. 利用結構化程式設計的概念來提升軟體開發之生產力

1.6 常見的資料結構 – 續

- 用來挑選資料結構的條件：
  1. 給定的資料量是相當大或是很小
  2. 資料是否需要常常增刪或更新，也就判斷資料是靜態的或是動態的？
  3. 資料在實際應用上被擷取次數之頻率
  4. 可用以儲存資料的記憶體容量有多大
  5. 實際應用時可以接受的資料擷取時間
  6. 使用某種資料結構是否很容易編寫對應程式碼